Example: StaticArrow
Static Arrow as defined in "Applicative Programming with effects" paper
Ref: https://www.staff.city.ac.uk/~ross/papers/Applicative.html

```
import Control.Applicative
import Control.Arrow
import qualified Control.Category as Cat


-- StaticArrow: as defined in "Applicative programming
with effects"
-- https://www.staff.city.ac.uk/~ross/papers/
Applicative.html
newtype StaticArrow f arr a b = StaticArrow
{getStaticArray :: f (arr a b)}

instance (Applicative f, Cat.Category an) =>
Cat.Category (StaticArrow f an) where
  id = StaticArrow $ pure Cat.id
  StaticArrow f . StaticArrow g = StaticArrow ((Cat..)
<$> f <*> g)

instance (Applicative f, Arrow an) => Arrow (StaticArrow
f an) where
  arr = StaticArrow . pure . arr
  first (StaticArrow f) = StaticArrow (first <$> f)

instance (Arrow an) => Cat.Category (WrappedArrow an)
where
  id = WrapArrow Cat.id
  f . g = WrapArrow (unwrapArrow f Cat.. unwrapArrow g)

instance (Arrow an) => Arrow (WrappedArrow an) where
  arr = WrapArrow . arr
  first f = WrapArrow (first $ unwrapArrow f)
```

Example: State Monad

Run

```
$ runhaskell TestStateMonad.hs
(-2,())
(8,())
(24,())
(24,"current state=24")
```

```
-- TestStateMonad.hs
import Control.Arrow

-- StateOp: represents an operation on the state.
--
```

```
-- s = current state
-- o = result of the operation
--
-- The state cannot be accessed directly, but only
through an operation,
-- including reading the current state.
-- e.g.
--    s -> (s,s)          read current state i.e. getOp
--    s -> (s+1,())       add 1 to the current state
--    s -> (10,())        reset the current state
--
--
--  Analogous to IO type
--  e.g.
--    -- IO
--    do
--      message <- getLine                  IO String
--      putStrLn $ "Hello " ++ message      IO ()
--
--    -- state
--    do
--      state <- getOp                      State Int
Int
--      putOp (state * 3)                   State Int ()
--
nullResult = ()
newtype StateOp s r = StateOp (s -> (s, r))

execOp :: StateOp s r -> s -> (s, r)
execOp (StateOp op) = op

-- write state operation
putOp :: s -> StateOp s ()
putOp state = StateOp $ const (state, nullResult)

-- get state operation
getOp :: StateOp s s
getOp = StateOp (\state -> (state, state))

-- map state operation
mapOp :: (s -> s) -> StateOp s ()
mapOp f = StateOp (\state -> (f state, nullResult))

multOp :: Num s => s -> StateOp s ()
multOp n = mapOp (*n)

addOp :: Num s => s -> StateOp s ()
addOp n = mapOp (+n)

instance Functor (StateOp s) where
  fmap f (StateOp op) = StateOp $ op >>> second f

instance Applicative (StateOp s) where
  pure x = StateOp (,x)
```

```
  StateOp getMap <*> StateOp getResult =
    StateOp
      ( \state ->
          let (_, resultMap) = getMap state
              (_, result) = getResult state
           in (state, resultMap result)
      )

instance Monad (StateOp s) where
  (StateOp execOp) >>= createOp =
    StateOp
      ( \state ->
          let (nextState, result) = execOp state
              (StateOp nextOp) = createOp result
           in nextOp nextState
      )

main = do
  let initState = (-1)
      toString = do
        state <- getOp
        return $ "current state=" ++ show state

  -- 3(2x + 10)
  print $ execOp (multOp 2) initState
  print $ execOp (do multOp 2; addOp 10) initState
  print $ execOp (do multOp 2; addOp 10; multOp 3)
initState
  print $ execOp (do multOp 2; addOp 10; multOp 3;
toString) initState
```